Multiway Search Tree.

-> Generalization of Binary tree

-> Number of keys stored per node
-> 1 (BST)
-> >1 (m-way search trees)

-> Number of max possible children is the order of the tree

-> No keys is 1 less than No of children

A m way search tree T may be an empty tree. If T is non empty, it satisfies the following prop

-> Each node has at most m children.
-> If a node has k child nodes ($k \le m$), then the node has exactly (k-1) keys.
-> The keys in each node are in ascending order
-> The keys in the left subtree of a key i are smaller than i
-> The keys in the right sub tree of a key i are greater than i.

Scanned with CamScanner

Insertion

Case-1

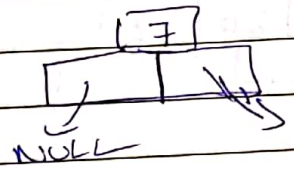
If a node can accomodate K, insert K in to the node and adjust the pointers

Case-2

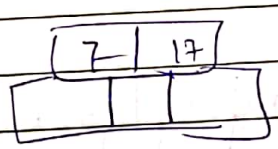
If a node cannot accomodate K, insert K into a new node at the next level.

ORDER -> 3

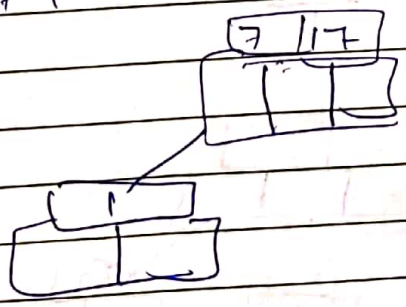
Insert 7



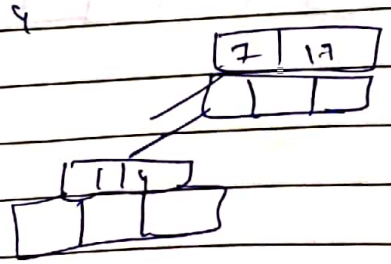
Insert 17



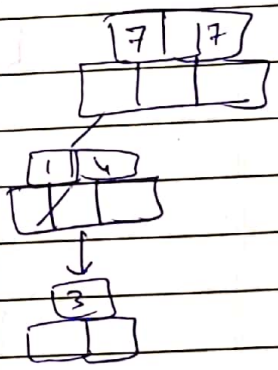
Insert 1



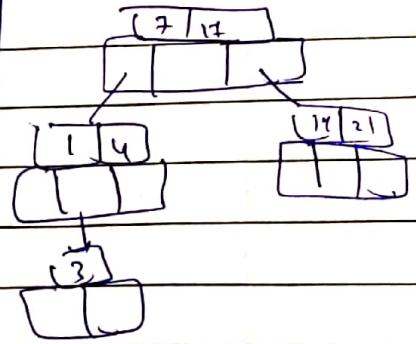
Insert 4



Insert 3.

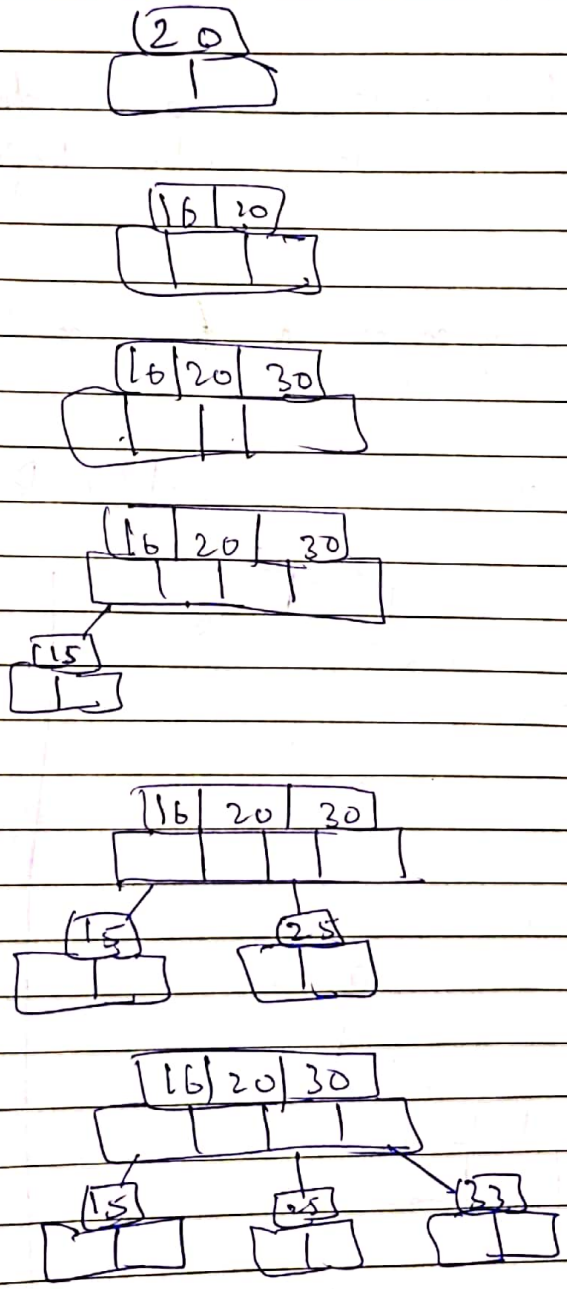


Insert 19

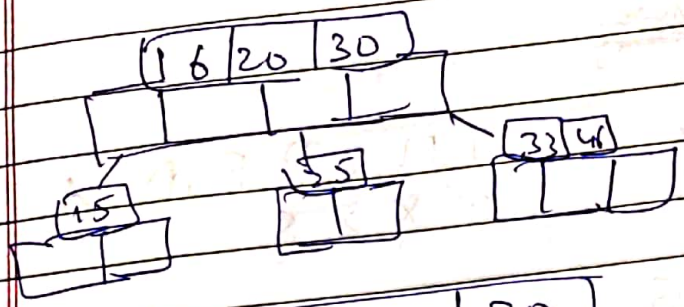


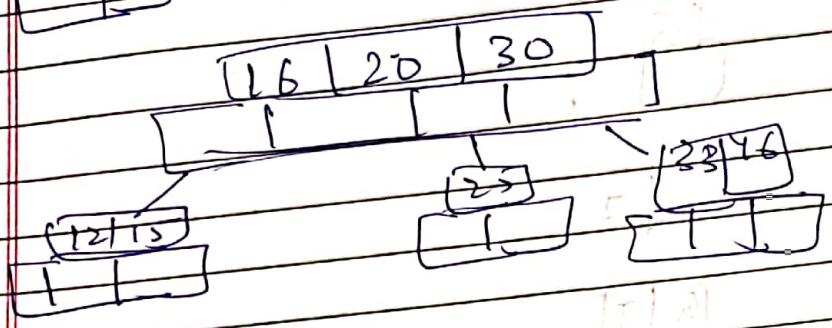
Scanned with CamScanner

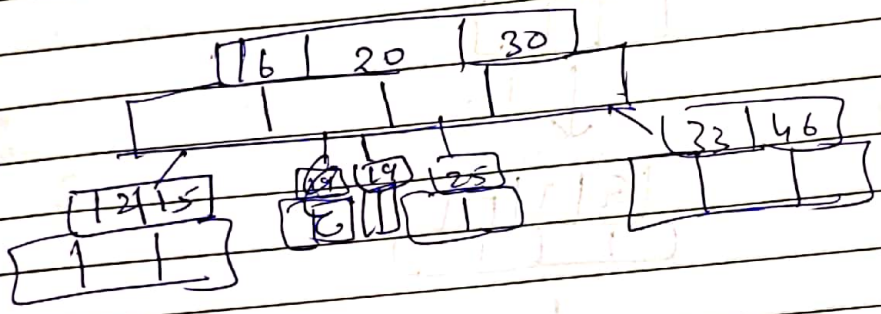
4-way	20	16	30	15	25	33
3 Keys	46	12	19	22	31	45

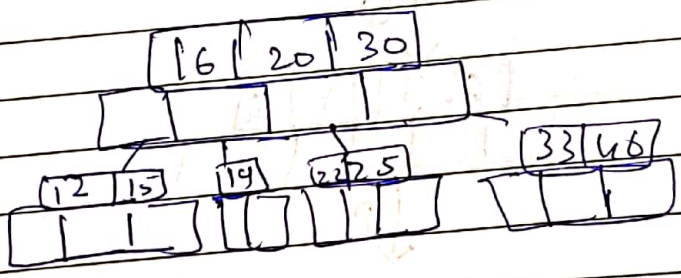


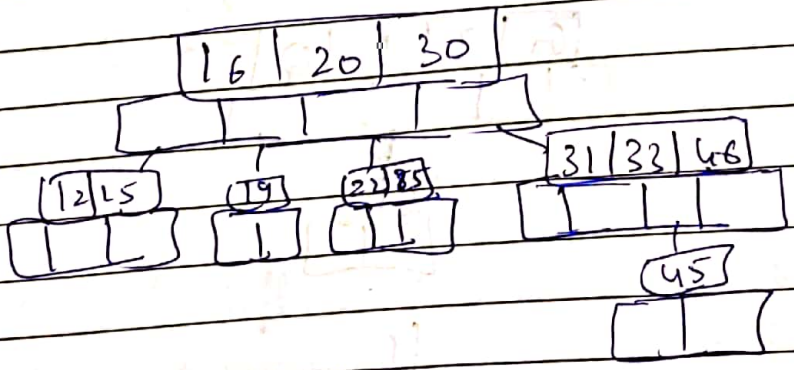
Scanned with CamScanner









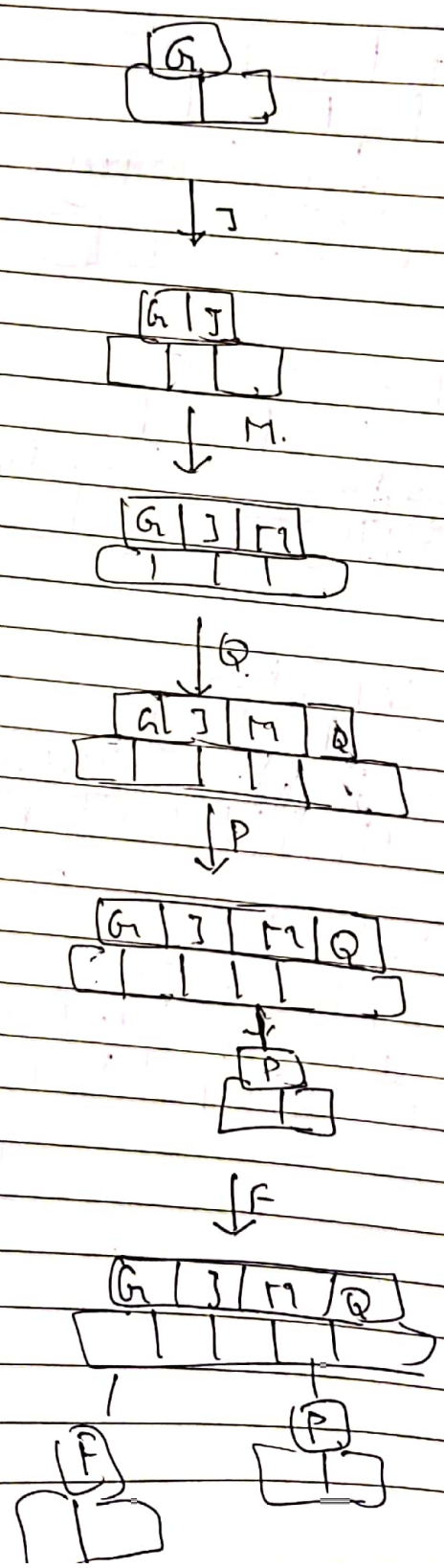


Scanned with CamScanner

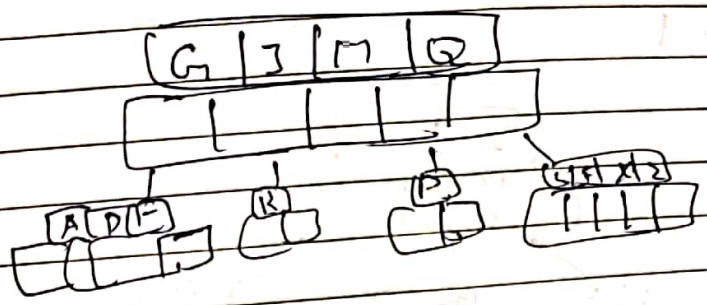
~~Deletion in Array~~

Insert 5 way 4 Keys

G, J, M, Q, P, F, D A K T X Z S

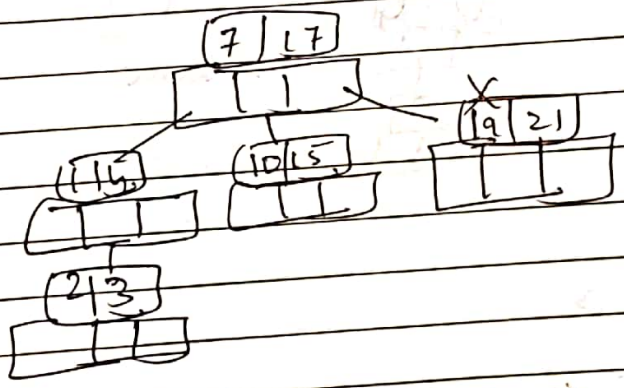


Scanned with CamScanner



Deletion

Case-1. If the left subtree of K are Null
Simply delete K and adjust the pointers

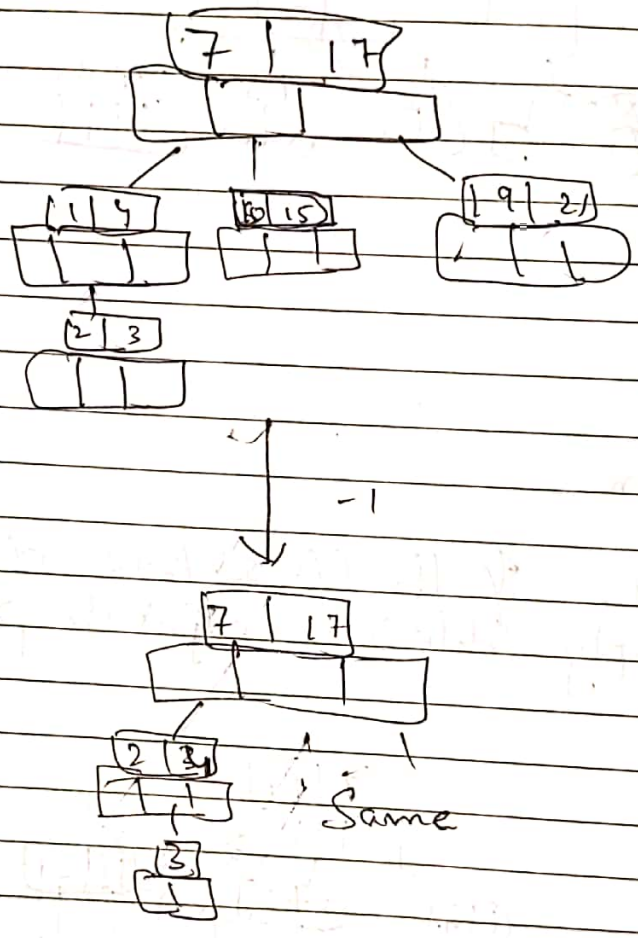


Case 2

If the left subtree of K is NULL and the right subtree is non-empty, choose the smallest key K' from the right subtree and replace K with K'

This may recursively call for deletion of K'

Delete 1

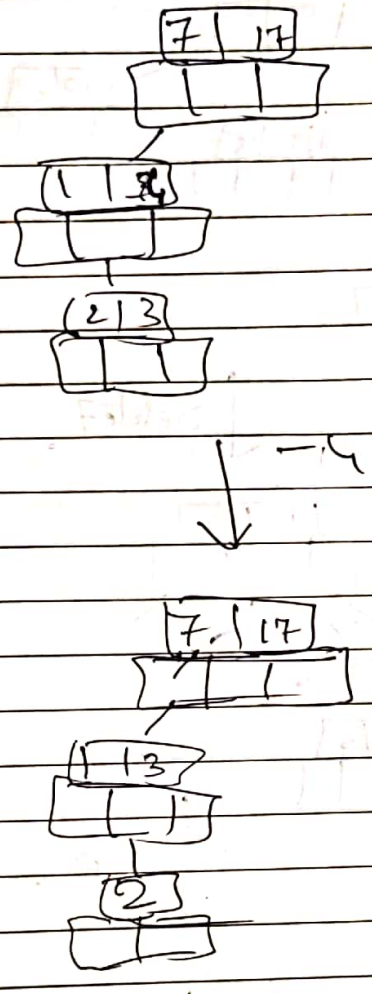


Case-3

If the right subtree of K is NULL and the left subtree is non-empty, choose the largest key K'' from the left subtree and replace K with K''.

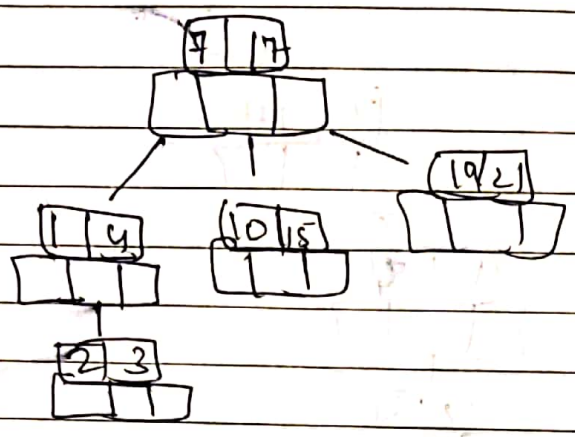
This may recursively call for K''

Delete 4

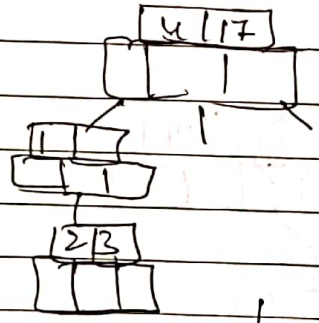


Case -4

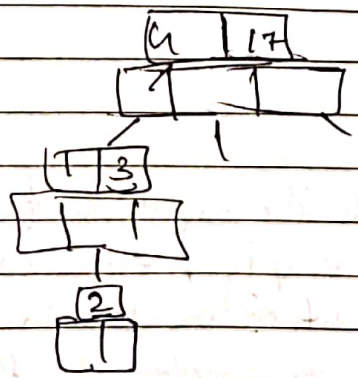
If both the right subtree and left subtree of K is non empty, either choose the largest key k'' from the left subtree or the smallest key k' from the right subtree and replace K with the chosen key

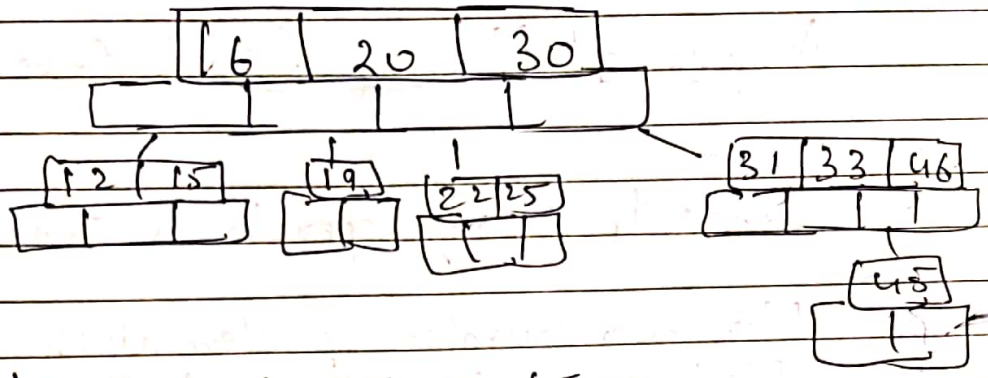


Delete 7.

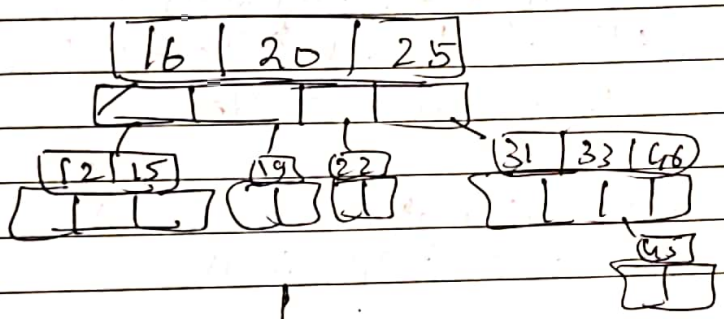


4 pos is empty

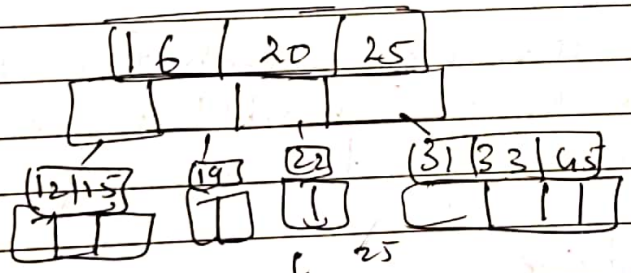




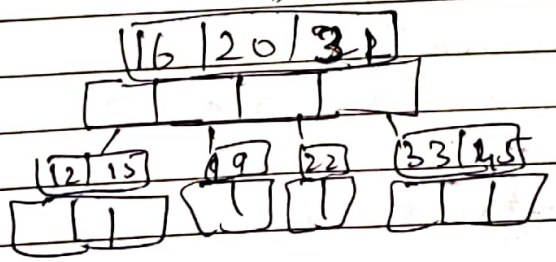
Delete 30, 46, 25, 19, 45, 20.



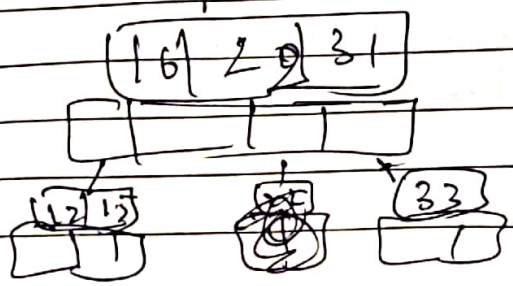
$\downarrow$ -46

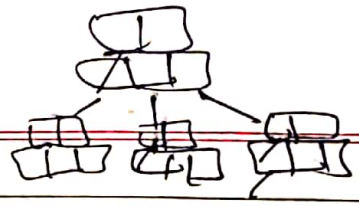


$\downarrow$ 25



-19, -45, -20





Analysis of M-way Search Trees

Complexity of any operation is $O(h)$

Minimum number of elements : h
Maximum number of elements.
At height 1 : $m-1$ Keys
At height 2 : $m(m-1)$ Keys
At h : $m^{h-1}(m-1)$ Keys.
Total number of elements : $m-1 + m(m-1) + \dots m^{h-1}(m-1)$

$(m-1)(1+m+m^2 \dots m^{h-1})$

$(m-1)(m^h-1)/(m-1)$

$= m^h - 1$

If a m-way search tree has n elements, the height h varies from a minimum of n to a max. of m^{h-1}

Best Case

$n = m^{h-1}$

$\log_m n = h-1 \log_m m$

$\log_m n + 1 = h.$

Worst Case : $O(h)$
Balancing is essential
B Trees.